

NETWORKING SECURE PROTOCOLS

[Kaynak](#)

İÇERİK

1. TLS (Transport Layer Security)
2. HTTPS (Hypertext Transfer Protocol Secure)
3. SMTPS, POP3S ve IMAPS (E-Posta Protokollerinde TLS)
4. SSH (Secure Shell)
5. SFTP ve FTPS (Güvenli Dosya Transferi)
6. VPN (Virtual Private Network - Sanal Özel Ağ)

TLS (Transport Layer Security)

Eskiden ağ üzerindeki trafiği izlemek (packet sniffing) çocuk oyuncağıydı. Bir saldırganın tek yapması gereken, ağ kartını **promiscuous mode** (karma mod - hedefi kendisi olmasa bile ağdaki tüm paketleri yakalama modu) durumuna getirmektir. Bu sayede tüm sohbetler, e-postalar ve en önemlisi parolalar kabak gibi ortadaydı; çünkü her şey **cleartext** (açık metin) olarak gönderiliyordu. Günümüzde ise parolaların şifrelenmeden gönderildiği bir servis görmek neredeyse imkansız. İşte bu dönüşümün mimarı **TLS** protokolüdür.

TLS'in Tarihsel Gelişimi ve Evrimi

Güvenli iletişim ihtiyacı 90'ların başında Netscape Communications tarafından fark edildi. Süreç SSL ile başladı ve bugün bildiğimiz modern TLS yapısına evrildi:

- **SSL 2.0 (1995):** Netscape tarafından yayınlanan ilk halka açık versiyon. (SSL - Secure Sockets Layer / Güvenli Yuva Katmanı).
- **TLS 1.0 (1999):** IETF (Internet Engineering Task Force) tarafından SSL 3.0'ın üzerine bir yükseltme olarak geliştirildi. (TLS - Transport Layer Security / Taşıma Katmanı Güvenliği).
- **TLS 1.3 (2018):** Protokolün tepeden tırnağa revize edildiği, güncel ve en güvenli versiyon.

Bu 20 yılı aşkın süreç bize şunu gösteriyor: Güvenlik statik bir şey değildir; her versiyon bir öncekinden alınan derslerle, kapatılan açıklarla ve iyileştirilen kriptografik yöntemlerle şekillenmiştir.

TLS Nedir ve Neden Hayatidir?

TLS, OSI modelinin **Transport Layer** (Taşıma Katmanı) üzerinde çalışan kriptografik bir protokoldür. Güvensiz bir ağ (örneğin halka açık bir Wi-Fi veya genel internet) üzerinden

istemci (client) ve sunucu (server) arasında güvenli bir tünel kurar.

Buradaki "güvenlik" kavramı iki ana temel direğe dayanır:

1. **Confidentiality (Gizlilik):** Veriyi gönderen ve alan dışındaki hiç kimsenin içeriği okuyamaması.
2. **Integrity (Bütünlük):** Verinin yolda giderken üçüncü şahıslar tarafından değiştirilmediğinden emin olunması.

Eğer TLS olmasaydı; online alışveriş, internet bankacılığı veya özel yazışmaların hiçbirini yapamazdık. Çünkü paketlerimizin yolda izlenmediğini veya manipüle edilmediğini asla garanti edemezdik.

"S" Takısı ve Modern Protokoller

Pek çok eski protokol, sonuna TLS eklenerek güvenli hale getirilmiştir. Genellikle ismin sonuna gelen **"S" (Secure)** harfi, o protokolün altında SSL/TLS çalıştığını gösterir:

- HTTP → HTTPS
- DNS → DoT (DNS over TLS)
- MQTT → MQTTS
- SIP → SIPS
- SMTP → SMTPS / POP3 → POP3S / IMAP → IMAPS

Teknik Arkaplan: Sertifika Mekanizması Nasıl İşler?

Bir sunucunun (veya bazen istemcinin) kimliğini doğrulaması için **imzalı bir TLS sertifikasına** ihtiyacı vardır. Bu süreç dijital dünyadaki "pasaport" veya "kimlik" çıkarma işlemine benzer.

Sertifika Alma Adımları:

1. **CSR (Certificate Signing Request) Oluşturma:** Sunucu yöneticisi bir Sertifika İmzalama İsteği oluşturur. Bu dosya sunucunun kimlik bilgilerini ve açık anahtarını içerir.
2. **CA (Certificate Authority) Onayı:** Bu CSR, güvenilir bir üçüncü taraf olan Sertifika Yetkilisine (CA) gönderilir. CA, isteği yapanın gerçekten o kişi/kurum olduğunu doğrular.
3. **Sertifika Yayını:** Doğrulama başarılıysa CA, kendi imzasıyla dijital sertifikayı oluşturur ve sunucuya iletir.

Güven Zinciri (Chain of Trust)

Bir cihazın (örneğin senin bilgisayarının), sunucudan gelen bu imzalı sertifikaya güvenebilmesi için, o sertifikayı imzalayan **CA'nın sertifikasının** senin cihazında zaten yüklü olması gerekir. İşletim sistemleri ve tarayıcılar, dünya çapında güvenilen otoritelerin (Root CA) sertifikalarını "Güvenilir Otoriteler" listesinde saklı tutar.

Önemli Not: Eskiden sertifika imzalatmak yıllık ücrete tabi ve pahalı bir işti. Ancak **Let's Encrypt** projesi sayesinde artık tamamen ücretsiz ve otomatik bir şekilde imzalı sertifikalar alınabiliyor.

Self-Signed (Kendinden İmzalı) Sertifikalar

Bazı durumlarda yöneticiler bir otoriteye gitmek yerine sertifikayı kendileri imzalarlar.

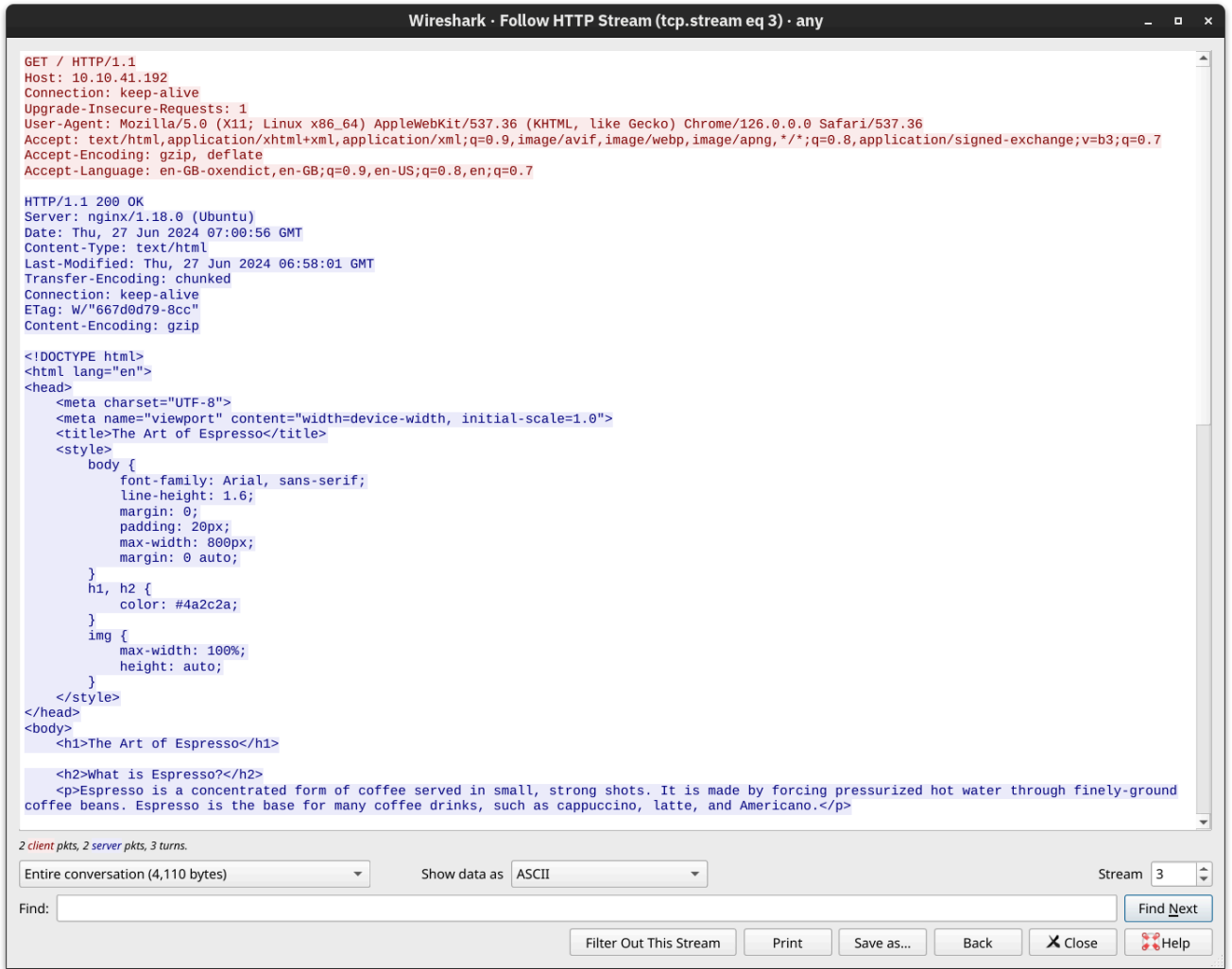
- **Dezavantaj:** Bu sertifika şifreleme sağlar (yani veriyi gizler), ancak **sunucunun kimliğini kanıtlayamaz**.
- **Sonuç:** Tarayıcıya girdiğinde "Bağlantınız gizli değil" uyarısı alıyorsan, o site muhtemelen kendinden imzalı veya süresi geçmiş bir sertifika kullanıyordur. Üçüncü bir taraf "Evet, bu site gerçekten iddia ettiği kişidir" diye imza atmadığı için tarayıcı haklı olarak şüpheyne düşer.

HTTPS (Hypertext Transfer Protocol Secure)

Bu bölümde, güvensiz olan **HTTP** protokolünün **TLS** ile nasıl zırhlandırıldığını ve bu iki yapı arasındaki temel farkları teknik detaylarıyla inceleyeceğiz.

1. Klasik HTTP: Neden Güvensiz?

HTTP, varsayılan olarak **TCP 80** portunu kullanır. En büyük zafiyeti, tüm trafiği **cleartext** (açık metin) olarak iletmesidir.



HTTP Bağlantı Aşamaları

Bir tarayıcı HTTP üzerinden bir sayfa istediğinde (DNS çözümlemesinden sonra) şu iki temel adımı izler:

1. **TCP Three-Way Handshake (Üçlü El Sıkışma):** İstemci ve sunucu arasında güvenilir bir taşıma yolu kurulur (**SYN**, **SYN-ACK**, **ACK** paketleri).
2. **HTTP İletişimi:** Bağlantı kurulur kurulmaz istemci `GET / HTTP/1.1` gibi isteklerini gönderir.

HTTP-espresso.pcapng						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
Apply a display filter ... <Ctrl-/>						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.11.81.126	10.10.41.192	TCP	76	52490 → 80 [SYN] Seq=0 Win=32120 Len=0 MSS=1460 SACK_PERM=1
2	0.073073828	10.10.41.192	10.11.81.126	TCP	76	80 → 52490 [SYN, ACK] Seq=0 Ack=1 Win=62643 Len=0 MSS=1460
3	0.073126006	10.11.81.126	10.10.41.192	TCP	68	52490 → 80 [ACK] Seq=1 Ack=1 Win=32120 Len=0 TSval=23161
4	0.073309310	10.11.81.126	10.10.41.192	HTTP	518	GET / HTTP/1.1
5	0.145362461	10.10.41.192	10.11.81.126	TCP	68	80 → 52490 [ACK] Seq=1 Ack=451 Win=62643 Len=0 TSval=376613
6	0.145416192	10.10.41.192	10.11.81.126	TCP	1344	80 → 52490 [ACK] Seq=1 Ack=451 Win=62643 Len=1276 TSval=376613
7	0.145450126	10.11.81.126	10.10.41.192	TCP	68	52490 → 80 [ACK] Seq=451 Ack=1277 Win=31872 Len=0 TSval=376613
8	0.145466797	10.10.41.192	10.11.81.126	HTTP	167	HTTP/1.1 200 OK (text/html)
9	0.145478078	10.11.81.126	10.10.41.192	TCP	68	52490 → 80 [ACK] Seq=451 Ack=1376 Win=31872 Len=0 TSval=376613
10	0.203973981	10.11.81.126	10.10.41.192	HTTP	459	GET /favicon.ico HTTP/1.1
11	0.276349438	10.10.41.192	10.11.81.126	HTTP	460	HTTP/1.1 404 Not Found (text/html)
12	0.276380436	10.11.81.126	10.10.41.192	TCP	68	52490 → 80 [ACK] Seq=842 Ack=1768 Win=31872 Len=0 TSval=376613
13	0.641397061	10.10.41.192	10.11.81.126	TCP	68	80 → 38892 [FIN, ACK] Seq=1 Ack=1 Win=482 Len=0 TSval=376613
14	0.641600272	10.11.81.126	10.10.41.192	TCP	68	38892 → 80 [FIN, ACK] Seq=1 Ack=2 Win=249 Len=0 TSval=376613
15	0.709830939	10.10.41.192	10.11.81.126	TCP	68	80 → 38892 [ACK] Seq=2 Ack=2 Win=482 Len=0 TSval=376613

Siber Güvenlik Açısından Analiz: Wireshark gibi bir paket yakalama aracıyla bu trafik izlendiğinde, saldırgan gönderilen tüm veriyi (parolalar, çerezler, form verileri) sanki bir gazete okuyormuş gibi net bir şekilde görebilir. Veri üzerinde hiçbir koruma katmanı yoktur.

2. HTTPS: HTTP'nin TLS Üzerindeki Hali

HTTPS, temel olarak **HTTP over TLS** demektir. Yani HTTP protokolü doğrudan TCP üzerine binmek yerine, araya bir güvenlik katmanı olan TLS'i alır.

HTTPS Bağlantı Aşamaları

HTTPS mimarisinde süreç bir adım daha uzar ama tam güvenlik sağlar:

1. **TCP Three-Way Handshake:** Önce taşıma katmanı kurulur (Port **443**).
2. **TLS Session Establishment (TLS El Sıkışması):** TCP kurulduktan hemen sonra başlar. Burada taraflar şifreleme algoritmalarında anlaşır, sunucu sertifikasını sunar ve şifreleme anahtarları oluşturulur.
3. **HTTP İletişimi:** Artık her şey hazır olduğunda, HTTP istekleri bu şifreli tünelin içinden gönderilir.

HTTPS-TryHackMe-Example.pcapng						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
Apply a display filter ... <Ctrl-/>						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.124.148	172.67.27.10	TCP	74	55680 → 443 [SYN] Seq=0 Win=32120
2	0.057774975	172.67.27.10	192.168.124.148	TCP	74	443 → 55680 [SYN, ACK] Seq=0 Ack=1
3	0.057860075	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=1 Ack=1 Win=
4	0.058522508	192.168.124.148	172.67.27.10	TLSv1.3	1821	Client Hello (SNI=tryhackme.com)
5	0.114467329	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=1 Ack=1756 W
6	0.120054588	172.67.27.10	192.168.124.148	TLSv1.3	5129	Server Hello, Change Cipher Spec,
7	0.120085857	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=1756 Ack=566
8	0.123398977	192.168.124.148	172.67.27.10	TLSv1.3	130	Change Cipher Spec, Application Da
9	0.123579706	192.168.124.148	172.67.27.10	TLSv1.3	158	Application Data
10	0.123768581	192.168.124.148	172.67.27.10	TLSv1.3	1771	Application Data
11	0.176342271	172.67.27.10	192.168.124.148	TLSv1.3	578	Application Data, Application Data
12	0.176342521	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=5576 Ack=361
13	0.176380392	172.67.27.10	192.168.124.148	TLSv1.3	97	Application Data
14	0.176446477	192.168.124.148	172.67.27.10	TCP	66	55680 → 443 [ACK] Seq=3617 Ack=566
15	0.176542286	192.168.124.148	172.67.27.10	TLSv1.3	97	Application Data
16	0.272679122	172.67.27.10	192.168.124.148	TCP	66	443 → 55680 [ACK] Seq=5607 Ack=364
17	0.272679613	172.67.27.10	192.168.124.148	TLSv1.3	533	Application Data
18	0.272679813	172.67.27.10	192.168.124.148	TLSv1.3	99	Application Data
19	0.272679994	172.67.27.10	192.168.124.148	TLSv1.3	1202	Application Data
20	0.272680164	172.67.27.10	192.168.124.148	TLSv1.3	1457	Application Data

Frame 1: 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on interface eth0, id 0
 Ethernet II, Src: 52:54:00:7c:d3:5b, Dst: 52:54:00:5d:c8:4d
 Internet Protocol Version 4, Src: 192.168.124.148, Dst: 172.67.27.10
 Transmission Control Protocol, Src Port: 55680, Dst Port: 443, Seq: 0, Len: 0

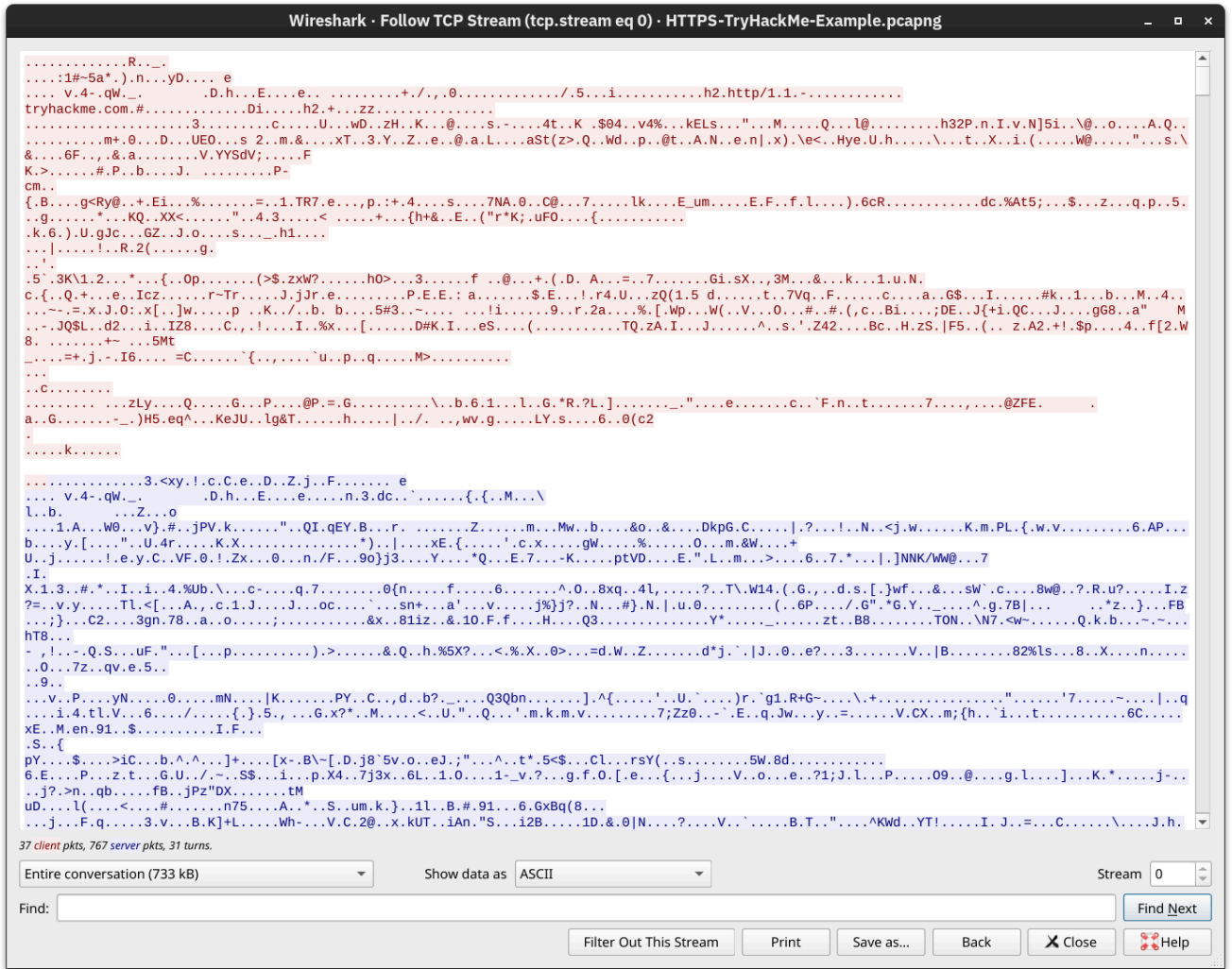
HTTPS-TryHackMe-Example.pcapng Packets: 1186 · Displayed: 1186 (100.0%) Profile: Default

Wireshark Gözlemi: Bir HTTPS trafiğini izlediğinizde, Wireshark paketleri artık "HTTP" olarak değil, "**Application Data**" (Uygulama Verisi) olarak etiketler. Çünkü paketler şifrelenmiştir ve analiz aracı bu verinin içinde bir HTTP isteği mi yoksa başka bir şey mi olduğunu (anahtara sahip değilse) bilemez.

3. Şifreli Trafiğin Analizi (Gibberish)

Eğer elinizde şifreleme anahtarı yoksa, yakaladığınız paket içeriği tamamen anlamsız karakterlerden (**gibberish**) oluşur.

- **Kırmızı metinler:** İstemci tarafından gönderilen veriyi,
- **Mavi metinler:** Sunucu tarafından verilen cevabı temsil eder.



Bu noktada saldırgan veriyi yakalasa bile, içindeki orijinal mesajı (örneğin bir GET isteğini veya bir POST parametresini) okuması matematiksel olarak imkansız yakındır.

4. Şifre Çözme (Getting the Encryption Key)

Bir siber güvenlik uzmanı veya sistem yöneticisi olarak, bazen kendi trafiğinizi hata ayıklama (debugging) amacıyla analiz etmeniz gerekebilir. Bunun için Wireshark'a sunucunun **Private Key** (Özel Anahtar) bilgisi veya **Session Keys** (Oturum Anahtarları) sağlanmalıdır.

Anahtar Girildiğinde Ne Değişir?

- **TCP ve TLS Handshake aynı kalır:** Bu aşamalar zaten şifreleme başlamadan önceki kurulum aşamalarıdır.
- **Veri Katmanı (Layer 3):** Wireshark artık "Application Data" yazan yerleri çözebilir ve içeriğin aslında bir GET /index.html isteği olduğunu size gösterebilir.

Kritik Çıkarım: TLS'in en büyük başarısı, ne alt katmandaki **TCP/IP** protokollerinde ne de üst katmandaki **HTTP** protokolünde hiçbir değişiklik gerektirmemesidir. TLS, bu iki dünya arasında şeffaf bir güvenlik köprüsü kurar.

SMTPTS, POP3S ve IMAPS (E-Posta Protokollerinde TLS)

E-posta protokollerine **TLS** eklemek, mantık olarak **HTTP**'yi **HTTPS**'e dönüştürmekten farksızdır. Temel amaç aynıdır: Verinin (e-postaların, kullanıcı adlarının ve parolaların) internet üzerinden açık metin (**cleartext**) olarak gitmesini engellemek.

E-Posta Protokolleri ve Güvenlik Dönüşümü

Tıpkı web trafiğinde olduğu gibi, e-posta iletişiminde kullanılan temel protokoller de sonuna bir **"S" (Secure)** takısı alarak TLS zırhına bürünürler. HTTPS için geçerli olan tüm teknik prensipler (TCP el sıkışması -> TLS el sıkışması -> Şifreli veri aktarımı) bu protokoller için de aynen geçerlidir.

Güvensiz (Cleartext) Versiyonlar

Bu protokollerin eski ve güvensiz versiyonları, veriyi herhangi bir şifreleme olmaksızın iletir. Bir saldırgan ağ trafiğini dinlediğinde e-postalarınızın içeriğini ve giriş bilgilerinizi rahatlıkla okuyabilir.

Protokol	Tanım	Varsayılan Port (TCP)
HTTP	Web Trafiği	80
SMTP	E-posta Gönderme (Simple Mail Transfer Protocol)	25
POP3	E-posta Alma/İndirme (Post Office Protocol v3)	110
IMAP	E-posta Erişimi/Senkronizasyon (Internet Message Access Protocol)	143

Güvenli (TLS ile Şifrelenmiş) Versiyonlar

TLS eklendiğinde bu protokoller yeni portlar üzerinden hizmet vermeye başlar. Bu port numaralarını bilmek, sızma testlerinde (penetration testing) veya ağ analizlerinde hangi servisin şifreli olduğunu anlamak için kritiktir.

Protokol	Tanım	Güvenli Port (TCP)
HTTPS	Güvenli Web	443
SMTPTS	Güvenli E-posta Gönderme	465 ve 587
POP3S	Güvenli E-posta Alma	995

Protokol	Tanım	Güvenli Port (TCP)
IMAPS	Güvenli E-posta Erişimi	993

Önemli Teknik Not: SMTPS ve Port Farklılıkları

SMTP protokolünde iki farklı güvenli port dikkati çeker:

- Port 465:** Orijinal olarak **Implicit TLS** (bağlantı kurulur kurulmaz TLS'in başlaması) için ayrılmıştır.
- Port 587:** Modern e-posta istemcilerinin (Outlook, Thunderbird vb.) e-posta göndermek için kullandığı standart porttur. Genellikle **STARTTLS** (bağlantı önce güvensiz başlar, sonra TLS'e yükseltilir) komutu ile güvenli hale getirilir.

Özet ve Mantık Zinciri

- Aynı Mekanizma:** HTTPS'te öğrendiğimiz "TCP + TLS + Uygulama Verisi" katman yapısı e-posta protokollerinde de birebir aynıdır.
- Şeffaflık:** TLS, e-posta içeriğine veya e-postanın nasıl gönderildiğine dair kurallara (SMTP komutları gibi) müdahale etmez; sadece bu verilerin geçtiği boruyu şifreler.
- Port Değişimi:** Bir servisi güvenli hale getirdiğimizde, genellikle trafiğin karışmaması için farklı bir port numarasına geçeriz (Örn: 143 -> 993).

Unutma: Bir sunucuda 110 veya 143 numaralı portların açık olduğunu görürsen, bu sunucunun eski nesil ve güvensiz e-posta trafiğine izin verdiğini, yani ortadaki adam (**MITM**) saldırılarına açık olduğunu anlayabilirsin.

SSH (Secure Shell)

Eskiden uzak sistemlere bağlanmak ve onları yönetmek için **TELNET** kullanılıyordu. Ancak TELNET'in çok büyük bir kusuru vardı: Tüm trafik (kullanıcı adları ve parolalar dahil) **cleartext** (açık metin) olarak iletiliyordu. Ağ trafiğini izleyen herhangi biri, giriş bilgilerini saniyeler içinde ele geçirebilirdi. Bu güvenlik açığı, 1995 yılında Tatu Ylönen'in **SSH** protokolünü geliştirmesine yol açtı.

İşin ilginç yanı; SSH-1'in çıktığı yıl (1995), Netscape'in SSL 2.0'ı çıkardığı yıllar aynıdır. Bugün kullandığımız modern SSH yapısı çoğunlukla 1999'da yayınlanan açık kaynaklı **OpenSSH** tabanlıdır.

SSH'in Sağladığı Temel Avantajlar

SSH sadece basit bir "uzaktan yönetim" aracı değildir; sunduğu güvenlik katmanları onu vazgeçilmez kılar:

- Güvenli Kimlik Doğrulama (Secure Authentication):** Sadece klasik parola değil, çok daha güvenli olan **Public Key** (Açık Anahtar) altyapısını ve iki faktörlü doğrulamayı (2FA)

destekler.

- **Gizlilik (Confidentiality):** Uçtan uca şifreleme sağlar. Kulak misafiri olan (eavesdropping) bir saldırgan veriyi göremez. Ayrıca sunucu anahtarı değiştiğinde sizi uyararak **Man-in-the-Middle** (Ortadaki Adam) saldırılarına karşı korur.
- **Bütünlük (Integrity):** Gönderilen verinin yolda değiştirilmediğini kriptografik yöntemlerle garanti eder.
- **Tünelleme (Tunneling):** Diğer güvensiz protokolleri SSH üzerinden yönlendirerek güvenli hale getirebilirsiniz. Bu özellik SSH'ı adeta bir **mini-VPN**'e dönüştürür.
- **X11 Forwarding:** Uzaktaki bir Unix sistemindeki grafik arayüzlü (GUI) uygulamaları, kendi yerel ekranınızda çalıştırmanıza olanak tanır.

SSH Bağlantı Komutları ve Parametreler

Bir SSH sunucusuna bağlanmak için temel komut yapısı şöyledir:

Bash

```
ssh username@hostname
```

- **username:** Uzak sunucudaki kullanıcı adınız.
- **hostname:** Sunucunun IP adresi veya domain adı.
- **Önemli:** Eğer yerel sistemdeki kullanıcı adınızla uzak sistemdeki aynıysa, sadece `ssh hostname` yazmanız yeterlidir.

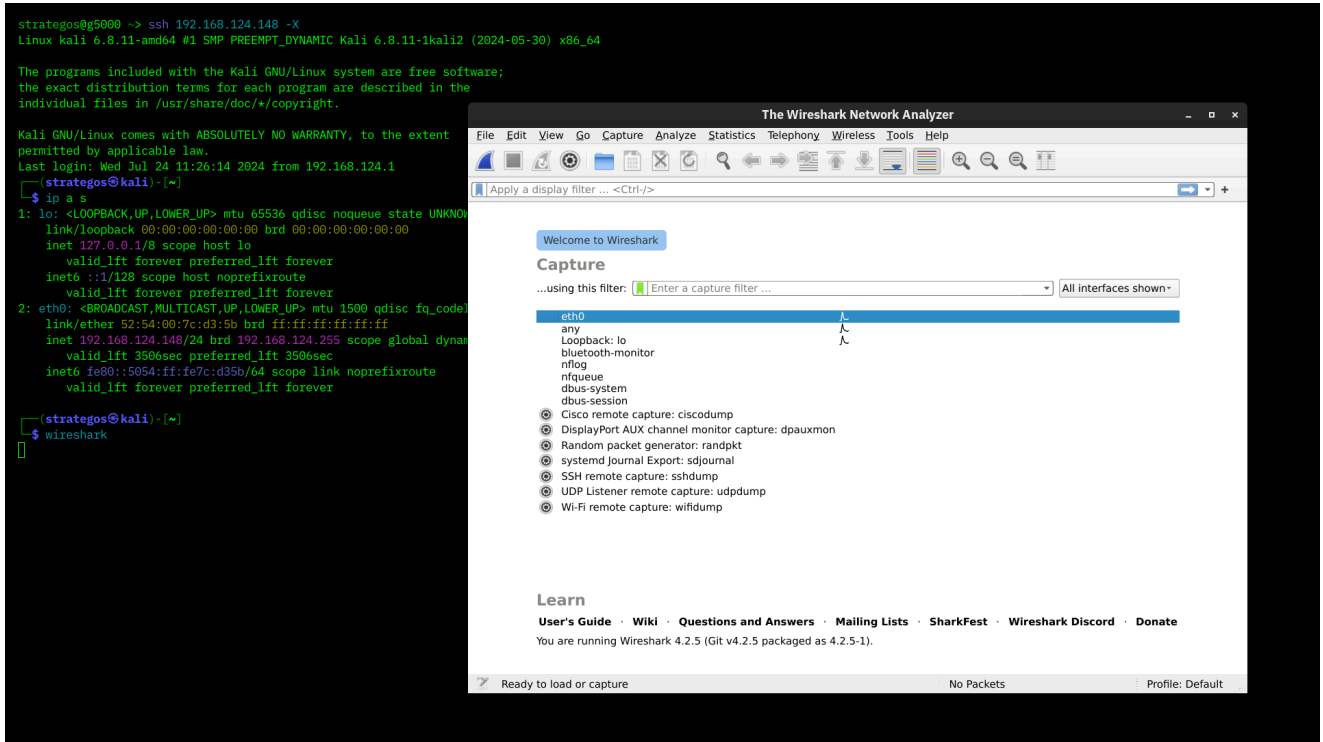
Grafik Arayüz Desteği (X11 Forwarding)

Eğer uzaktaki bir aracın (örneğin Wireshark veya Firefox) grafik arayüzünü kendi ekranınızda görmek istiyorsanız `-X` parametresini kullanmalısınız:

Bash

```
ssh 192.168.124.148 -X
```

- **-X:** X11 yönlendirmesini etkinleştirir. Yerel bilgisayarınızda bir X-Server (Linux'ta yerleşiktir, Windows için VcXsrv veya Xming gerekir) yüklü olmalıdır.



Port Karşılaştırması

Ağ taramalarında (Nmap vb.) bu iki protokolü portlarından hemen ayırt edebiliriz:

Protokol	Port	Güvenlik Durumu
TELNET	23	Güvensiz (Şifreleme yok)
SSH	22	Güvenli (Şifreli)

Önemli Bir Mantık Notu:

SSH, TLS protokolünü kullanmaz. Kendi içinde farklı bir şifreleme ve anahtar değişim mekanizması vardır. Ancak sağladığı sonuç (gizlilik ve bütünlük) TLS ile benzerdir.

SFTP ve FTPS (Güvenli Dosya Transferi)

Dosya transferi dendiğinde akla ilk gelen protokol **FTP** (File Transfer Protocol) olsa da, FTP'nin verileri ve parolaları şifrelemeden göndermesi büyük bir güvenlik açığıdır. Bu sorunu çözmek için iki farklı yaklaşım geliştirilmiştir: **SFTP** ve **FTPS**. İsimleri çok benzer olsa da, çalışma mantıkları ve kullandıkları altyapılar tamamen farklıdır.

1. SFTP (SSH File Transfer Protocol)

SFTP, adından da anlaşılacağı üzere **SSH** protokol ailesinin bir parçasıdır. Ayrı bir protokol olmaktan ziyade, güvenli bir SSH tüneli üzerinden dosya aktarımı yapılmasına olanak tanır.

- **Port:** SSH ile aynı portu, yani **TCP 22** portunu kullanır.

- **Yapılandırma:** Bir **OpenSSH** sunucusunda SFTP'yi aktif etmek son derece kolaydır; sadece yapılandırma dosyasındaki bir seçeneğin açılması yeterlidir.
- **Kullanım:** Terminal üzerinden şu komutla bağlanılır: Bash

```
sftp username@hostname
```

- **Temel Komutlar:** SFTP komut satırı Unix benzeri bir yapıdadır:
 - `get dosya_adi` : Sunucudan kendi makinenize dosya **indirir**.
 - `put dosya_adi` : Kendi makinenizden sunucuya dosya **yükler**.
 - `ls` , `cd` , `pwd` : Dosya sisteminde gezinmek için kullanılır.

2. FTPS (File Transfer Protocol Secure)

FTPS, bildiğimiz klasik **FTP** protokolünün üzerine **TLS** (veya SSL) katmanı eklenmiş halidir. Mantığı HTTPS, SMTPS veya IMAPS ile tamamen aynıdır: Mevcut protokolü al ve bir TLS tüneli içine sok.

- **Port:** Standart FTP portu 21 iken, **FTPS** genellikle **990** numaralı portu kullanır.
- **Sertifika Gereksinimi:** Tıpkı bir web sunucusunda olduğu gibi, FTPS sunucusunun da çalışması için geçerli bir **TLS Sertifikasına** ihtiyacı vardır.
- **Zorluklar:** FTPS, kontrol sinyalleri ve veri aktarımı için ayrı bağlantılar kullandığından, katı güvenlik duvarı (firewall) kuralları olan ağlarda yapılandırması bazen sancılı olabilir.

Karşılaştırma Tablosu: Aralarındaki Farklar Neler?

Özellik	SFTP	FTPS
Tam Adı	SSH File Transfer Protocol	FTP over TLS/SSL
Altyapı	SSH tabanlıdır.	FTP + TLS tabanlıdır.
Port	Varsayılan 22	Varsayılan 990 (Bazen 21 + STARTTLS)
Sertifika	SSH Anahtarları kullanılır.	X.509 Dijital Sertifika (CA imzalı) gerekir.
Firewall Dostu	Çok dostane (Tek bir port yeterli).	Karmaşık (Veri ve Kontrol kanalları farklıdır).

VPN (Virtual Private Network - Sanal Özel Ağ)

Dünyanın farklı yerlerinde ofisleri olan bir şirket düşün. Bu ofislerdeki tüm cihazların, sanki ana merkezdeymiş gibi dosya paylaşımına ve kaynaklara erişmesi gerekir. Bunu fiziksel kablolarla yapmak imkansız ve maliyetlidir. İşte burada devreye internet altyapısını kullanan, ancak trafiği gizli tutan **VPN** girer.

VPN ismindeki iki harf, bu teknolojinin ruhunu özetler:

- **V (Virtual - Sanal):** Fiziksel bir hat çekilmez; mevcut internet altyapısı üzerinde "sanal" bir bağ kurulur.
- **P (Private - Özel):** İnternet trafiği herkesin görebileceği şekilde akarken, VPN bu trafiği şifreleyerek sadece yetkili tarafların okuyabileceği "özel" bir tünele hapseder.

VPN Neden Gereklidir? (Güvenlik Açığı ve Çözüm)

İnternetin temeli olan **TCP/IP** protokol takımı tasarlanırken odak noktası "verinin hedefe ulaşmasıydı", "verinin gizliliği" değil.

- Bir yönlendirici (router) bozulursa, protokoller farklı bir yol bulur.
- Bir paket kaybolursa, TCP bunu fark eder ve tekrar gönderir.
- **Ancak:** Verinin yolda okunmasını veya değiştirilmesini engelleyecek dahili bir mekanizma yoktur.

VPN, bu eksikliği gidermek için tüm ağ trafiğini bir **şifreleme tüneli** içine alır.

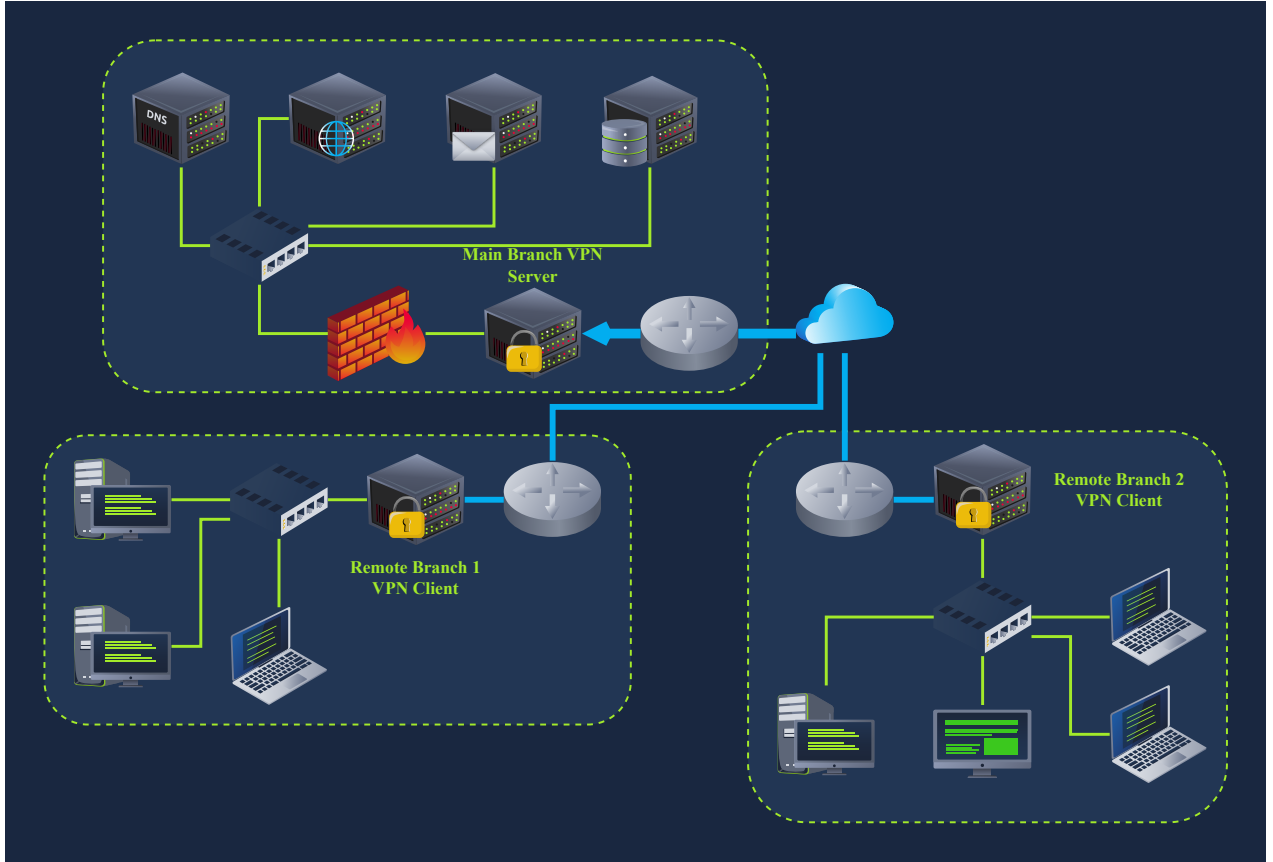
VPN Çalışma Mantığı ve Ağ Yapısı

VPN kurulumu için temel olarak bir **VPN Sunucusu (Server)** ve bir **VPN İstemcisi (Client)** gerekir. Trafik akışı şu şekilde gerçekleşir:

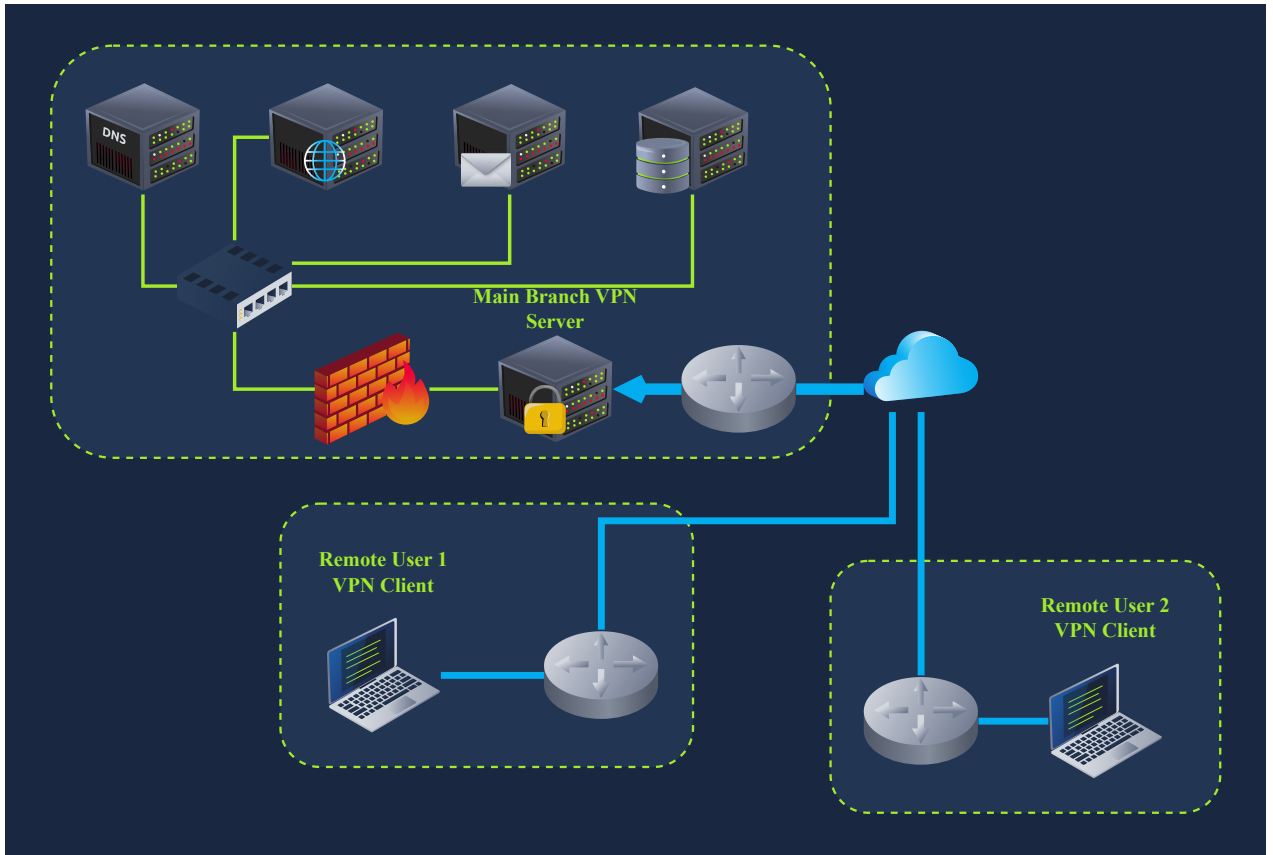
1. **Şifreleme:** VPN istemcisi veriyi henüz cihazdan çıkmadan şifreler.
2. **Tünelleme:** Şifrelenmiş veri, internet üzerinden bir "tünel" (mavi hat) aracılığıyla VPN sunucusuna iletilir.
3. **Çözme (Decryption):** VPN sunucusu veriyi alır, şifresini çözer ve hedef sisteme iletir.
4. **İç Ağ Erişimi:** Bu aşamadan sonra veri, şirketin iç ağında (yeşil hat) sanki o cihaz oradaymış gibi dolaşabilir.

Kullanım Senaryoları

- **Siteden Siteye (Site-to-Site):** İki uzak ofisin (şube) ana merkeze bağlanması.



- **Uzaktan Erişim (Remote Access):** Evden çalışan bir kullanıcının tek bir cihazla şirket ağına bağlanması.



VPN'in Kullanıcıya Sağladığı Avantajlar

VPN bağlandığında, genellikle tüm internet trafiği bu tünel üzerinden geçer. Bunun birkaç önemli sonucu vardır:

- **IP Gizleme:** İnternetteki servisler sizin gerçek IP adresinizi değil, VPN sunucusunun IP adresini görür.
- **Coğrafi Kısıtlamaları Aşma:** Örneğin, Japonya'daki bir VPN sunucusuna bağlanırsanız, Google sizi Japonya'da sanır ve aramalarınızı buna göre özelleştirir. Bölgesel engelli içeriklere bu şekilde erişilebilir.
- **ISP Sansüründen Kaçınma:** Yerel internet servis sağlayıcınız (ISP) sadece şifreli bir trafik görür. Hangi siteye girdiğinizi veya ne indirdiğinizi bilemediği için içerik bazlı sansür uygulayamaz.

Kritik Uyarılar ve Teknik Riskler

Her VPN bağlantısı %100 gizlilik veya her yere erişim anlamına gelmez:

- **Yapılandırma Farkları:** Bazı VPN'ler sadece belirli bir ağa erişim sağlar, tüm internet trafiğinizi tünellemez.
- **DNS Sızıntısı (DNS Leak):** En tehlikeli durumlardan biridir. Trafiğiniz şifreli olsa bile, hangi siteye gitmek istediğiniz bilgisinin (DNS sorgusu) gerçek ISP'niz üzerinden gitmesi durumudur. Bu yüzden her zaman bir **DNS Leak Test** yapılması önerilir.
- **Yasal Durum:** Bazı ülkelerde VPN kullanımı yasa dışıdır veya kısıtlanmıştır. Seyahat ederken yerel yasaları kontrol etmek kritiktir.